

Consolidated Oneliner Library

GULECD-UFPR

May 3, 2026

Contents

1	Postgresql	3
1.1	Vacuum Verbose Analyze	3
2	Python	6
2.1	Csv To Tsv	6
2.2	Tsv To Csv	8
3	Bash	11
3.1	Bulk Create Users From Csv	11
3.2	Bulk Pgp Detach Sign Files	13
3.3	Create Fallocate	16
3.4	Disk Space Sort Largest Directories	18
3.5	Disk Space Usage Per Directory	20
3.6	Dmesg Errors Pretty	23
3.7	Find 10 Largest Storage Users	26
3.8	Find Large Files Recursive	29
3.9	Journalctl Errors Today	31
3.10	List All Aliases	33
3.11	Nonhuman User Activity And Quota Report	36
3.12	Pretty Print Public Ssh Pgp Keys	39
3.13	Replace String In Files	42
3.14	System Memory Report Top10	45
3.15	User Activity And Quota Report	47
4	R	52
4.1	Csv Stats	52
4.2	Tsv Stats	54
5	Git	57
5.1	Init Repo With Gitignore	57

Chapter 1

Postgresql

1.1 Vacuum Verbose Analyze

Perform PostgreSQL VACUUM with VERBOSE and ANALYZE options

Author: Marcos de Carvalho

Date: 2026-04-25

Executes PostgreSQL VACUUM operation with VERBOSE output and ANALYZE for statistics update, following best practices for database maintenance. Accepts arguments: username password database host.

Language

postgresql

Category

database

Command

```
PGPASSWORD="$2" psql -U "$1" -d "$3" -h "$4" -c "VACUUM (VERBOSE, ANALYZE);"
```

Explanation

The command connects to PostgreSQL via psql and runs VACUUM with VERBOSE (detailed output) and ANALYZE (update statistics). VACUUM reclaims storage occupied by dead tuples, while ANALYZE updates planner statistics. This combination is recommended for routine maintenance. Arguments: \$1=username, \$2=password (sets PGPASSWORD), \$3=database, \$4=host.

Tags

postgresql, database, maintenance, vacuum, optimization

Dependencies

postgresql-client

Arguments

1. **USERNAME** (Required): PostgreSQL username
2. **PASSWORD** (Required): PostgreSQL password (sets PGPASSWORD environment variable)
3. **DATABASE** (Required): Database name
4. **HOST** (Optional): Database host
Default: localhost

Examples

1. `vacuum_pg() { PGPASSWORD="$2" psql -U "$1" -d "$3" -h "$4" -c "VACUUM (VERBOSE, ANALYZE)"; }` - Define shell function for alias usage
2. `vacuum_pg postgres secret123 mydb localhost` - Call function with arguments: user password db host
3. `PGPASSWORD="secret456" psql -U "appuser" -d "production" -h "db.example.com" -c "VACUUM (VERBOSE, ANALYZE);"` - Direct command with explicit values

Output

```
VACUUM
INFO:  vacuuming "public.some_table"
INFO:  "some_table": removed 1000 row versions in 5 pages
INFO:  "some_table": found 1000 removable, 5000 nonremovable row versions
DETAIL:  0 dead row versions cannot be removed yet.
CPU 0.00s/0.00u sec elapsed 0.00 sec.
INFO:  analyzing "public.some_table"
INFO:  "some_table": scanned 3000 of 3000 pages, containing 5000 live rows and 0
        dead rows
VACUUM
```

Notes

- Run during periods of low database activity
- VERBOSE provides detailed output for monitoring
- ANALYZE updates statistics for query planner
- Consider using .pgpass file for password management
- Monitor pg_stat_activity for concurrent sessions
- For alias usage: `vacuum_pg() { PGPASSWORD="$2" psql -U "$1" -d "$3" -h "$4" -c "VACUUM (VERBOSE, ANALYZE)"; }`

Warnings

- `VACUUM` requires `ACCESS SHARE` lock on tables (conflicts with DDL)
- Avoid running during peak hours or critical operations
- For aggressive cleanup, consider `VACUUM FULL` but it requires exclusive lock
- Ensure adequate `maintenance_work_mem` setting for performance

See Also

- `postgresql-reindex`
- `postgresql-analyze-only`

Status

draft

Safety

caution

Shell

bash

Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

Created At

2026-04-25

Updated At

2026-04-25

Chapter 2

Python

2.1 Csv To Tsv

Convert CSV file to TSV (tab-separated values) using Python stdlib

Author: Marcos de Carvalho

Date: 2026-04-25

Converts comma-separated values (CSV) to tab-separated values (TSV) using Python's standard library csv module.

Language

python

Category

data-processing

Command

```
python3 -c "import sys, csv; csv.writer(sys.stdout, delimiter='\t').writerows(csv.reader(sys.stdin))"
```

Explanation

The command reads CSV data from stdin, parses it with csv.reader (default delimiter is comma), and writes it to stdout using csv.writer with tab delimiter. It preserves quoting and escaping rules.

Tags

python, csv, tsv, data-conversion, text-processing

Dependencies

python3

Arguments

1. **INPUT_FILE** (Optional): CSV input file (use stdin redirection)
Default: stdin
2. **OUTPUT_FILE** (Optional): TSV output file (use stdout redirection)
Default: stdout

Examples

1. `python3 -c "import sys, csv; csv.writer(sys.stdout, delimiter='\\t').writerows(csv.reader(sys.stdin, delimiter='\\t'))"` - Convert data.csv to data.tsv
2. `cat data.csv | python3 -c "import sys, csv; csv.writer(sys.stdout, delimiter='\\t').writerows(csv.reader(sys.stdin, delimiter='\\t'))"` - Preview first 20 rows converted

Output

col1	col2	col3
value1	value2	value3

Notes

- Handles quoted fields with commas correctly
- Preserves newlines within quoted fields
- Works with any CSV dialect compatible with Python's csv module

Warnings

- Large files may consume memory as all rows are read into list
- Input must be valid CSV format

See Also

- `tsv-to-csv`

Status

reviewed

Safety

safe

Shell

posix

Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

Created At

2026-04-25

Updated At

2026-04-25

2.2 Tsv To Csv

Convert TSV (tab-separated values) file to CSV using Python stdlib

Author: Marcos de Carvalho

Date: 2026-04-25

Converts tab-separated values (TSV) to comma-separated values (CSV) using Python's standard library csv module.

Language

python

Category

data-processing

Command

```
python3 -c "import sys,csv; csv.writer(sys.stdout).writerows(csv.reader(sys.stdin, delimiter='\t'))"
```

Explanation

The command reads TSV data from stdin, parses it with csv.reader using tab delimiter, and writes it to stdout using csv.writer with default comma delimiter. It preserves quoting and escaping rules.

Tags

python, tsv, csv, data-conversion, text-processing

Dependencies

python3

Arguments

1. **INPUT_FILE** (Optional): TSV input file (use stdin redirection)
Default: stdin
2. **OUTPUT_FILE** (Optional): CSV output file (use stdout redirection)
Default: stdout

Examples

1. `python3 -c "import sys, csv; csv.writer(sys.stdout).writerows(csv.reader(sys.stdin, delimiter='\\{\\t', < data.tsv > data.csv` - Convert data.tsv to data.csv
2. `cat data.tsv | python3 -c "import sys, csv; csv.writer(sys.stdout).writerows(csv.reader(sys.stdin, delimiter='\\{\\t'))" | head -20` - Preview first 20 rows converted

Output

```
col1,col2,col3
value1,value2,value3
```

Notes

- Handles quoted fields with tabs correctly
- Preserves newlines within quoted fields
- Works with any TSV dialect compatible with Python's csv module

Warnings

- Large files may consume memory as all rows are read into list
- Input must be valid TSV format

See Also

- `csv-to-tsv`

Status

reviewed

Safety

safe

Shell

posix

Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

Created At

2026-04-25

Updated At

2026-04-25

Chapter 3

Bash

3.1 Bulk Create Users From Csv

Create multiple users from a CSV file containing username and password

Author: marcos

Date: 2026-04-25

Creates multiple user accounts on a Linux system by reading username and password pairs from a CSV file passed as an argument.

Language

bash

Category

system-administration

Command

```
sudo awk -F, 'NR>1 {print $1,$2}' "$1" | while read user pass; do sudo useradd -m "$user" && echo "$user:$pass" | sudo chpasswd; done
```

Explanation

The command reads a CSV file (passed as first argument) where each row contains a username and password. It skips the header row (`NR<1`), then for each subsequent row extracts the username and password fields. For each user, it creates a system account with a home directory (`-m`) and then uses `chpasswd` to set the plain text password from the CSV file.

Tags

user-management, bulk-operations, system-administration, csv, automation

Dependencies

coreutils, shadow-utils, openssl

Arguments

1. **CSV_FILE** (Required): Path to CSV file with username,password format

Examples

1. `echo -e 'username,password\{ }nuser1,pass123\{ }nuser2,pass456' > users.csv && sudo awk -F, 'NR>1 {print $1,$2}' "users.csv" | while read user pass; do sudo useradd -m -p $(openssl passwd -1 "$pass") "$user"; done` - Create users from users.csv file with username,password format
2. `sudo awk -F, 'NR>1 {print $1,$2}' "/path/to/users.csv" | while read user pass; do sudo useradd -m -p $(openssl passwd -1 "$pass") "$user"; done` - Create users from a specific CSV file path
3. `alias bulkaddusers='sudo awk -F, '""'NR>1 {print $1,$2}'""' "$1" | while read user pass; do sudo useradd -m -p $(openssl passwd -1 "$pass") "$user"; done'` - Create an alias for bulk user creation

Output

```
Creating users from users.csv:
user1: user created
user2: user created
```

Notes

- The CSV file should have a header row and contain username,password in each subsequent row
- Requires sudo privileges to create users
- Uses openssl to properly hash passwords for useradd
- The -m flag creates a home directory for each user
- The command skips the first line (header) with NR<1
- For better security, consider using a more robust password hashing method
- The CSV file must be passed as the first argument to the command

Warnings

- This command requires sudo access and will modify system user accounts
- Passwords are passed in cleartext in the CSV file which is a security risk
- The default password hashing method (openssl passwd -1) uses MD5 which is not the most secure
- Always backup your system before running bulk user creation

- Ensure CSV file permissions are secured to prevent unauthorized access
- Test with a single entry first before running on multiple users
- Consider using a more secure method for password handling in production environments

See Also

- user-activity-and-quota-report
- nonhuman-user-activity-and-quota-report

Status

draft

Safety

caution

Shell

bash

Platforms

linux, gnu-linux

Created At

2026-04-25

Updated At

2026-04-25

3.2 Bulk Pgp Detach Sign Files

Create detached PGP signatures for all files in a directory

Author: Marcos de Carvalho

Date: 2026-04-28

Iterates over every file in a given directory and creates a detached PGP signature (.sig) for each one using GnuPG.

Language

bash

Category

cryptography

Command

```
bash -c 'dir=$1; shift; for f in "$dir"/*; do [ -f "$f" ] && gpg --detach-sign "$f"; done' _
```

Explanation

This oneliner uses a `bash -c` subshell to accept the target directory as a trailing argument. The variable `dir` receives `$1` (the directory path), then `shift` removes it so remaining arguments are available via `$@`. A `for` loop iterates over all entries matching `"$dir"/*`, and `[ -f "$f" ]` ensures only regular files are processed (skipping subdirectories, symlinks, etc.). For each file, `gpg --detach-sign` creates a separate `.sig` file containing the PGP signature. The `_` placeholder serves as `$0` inside the subshell, ensuring `$1` maps to the first real argument.

Tags

gpg, pgp, cryptography, signature, detach-sign, bulk, verification

Dependencies

gnupg

Arguments

1. **DIR** (Optional): Path to the directory whose files will be signed.
Default: `.`

Examples

1. `bash -c 'dir=$1; shift; for f in "$dir"/*; do [ -f "$f" ] && gpg --detach-sign "$f"; done' _ /path/to/releases` - Sign all files in `/path/to/releases`
2. `bash -c 'dir=$1; shift; for f in "$dir"/*; do [ -f "$f" ] && gpg --detach-sign "$f"; done' _ .` - Sign all files in the current directory
3. `alias bsign="bash -c 'dir=\{}$1; shift; for f in \{"\}$dir\{"\}/*; do [ -f \{"\}\$f\{"\} ] && gpg --detach-sign \{"\}\$f\{"\}"; done' _"` - Create an alias named `'bsign'` for easy bulk signing
4. `bsign /path/to/releases` - Use the alias to sign all files in a directory

Output

```
file1.tar.gz.sig
file2.iso.sig
file3.deb.sig
```

Notes

- Each original file gets a corresponding .sig file in the same directory
- GPG will prompt for passphrase if the private key is passphrase-protected
- Uses the default signing key; set GPGKEY env var or use `-default-key` to specify another
- The `[ -f "$f" ]` check skips subdirectories and non-regular files
- Hidden files (dotfiles) are not matched by the glob `*`; use `.*` as well if needed
- For large numbers of files, consider using `gpg -batch -yes` to skip confirmation prompts

Warnings

- Existing .sig files will be silently overwritten without confirmation
- Ensure you have sufficient disk space for the signature files
- If GPG cannot access your private key, the loop will fail for every file
- Signatures are created in the same directory as the source files

See Also

- `find-large-files-recursive`

Status

reviewed

Safety

caution

Shell

bash

Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

Created At

2026-04-28

Updated At

2026-04-28

3.3 Create Fallocate

Create a large file of a specific size quickly

Author: Marcos de Carvalho

Date: 2026-04-21

Quickly creates a file of a specific size (e.g., 1 GB) using fallocate, which is much faster than dd because it allocates blocks without writing actual data.

Language

bash

Category

filesystem

Command

```
fallocate -l 1G test_file
```

Explanation

The 'fallocate' command communicates directly with the filesystem to preallocate disk space for a file. Unlike 'dd', which must write every byte to disk, 'fallocate' simply marks the space as allocated in the filesystem's metadata. This is extremely efficient for creating large test files, swap files, or preallocating space for databases. The length (-l) can be specified in bytes or using suffixes like K, M, G, T, P, E.

Tags

fallocate, filesystem, storage, disk-usage, allocation, testing

Dependencies

util-linux

Arguments

1. **SIZE** (Optional): The size of the file to create (e.g., 10M, 1G, 500M).
Default: 1G
2. **FILENAME** (Optional): The name of the file to be created.
Default: test_file

Examples

1. `fallocate -l 100M dummy_file` - Create a 100 MB file named dummy_file
2. `fallocate -l 1G large_test_file` - Create a 1 GB file for testing

3. `fallocate -l 10G /swapfile` - Preallocate 10 GB for a swap file (requires subsequent `mkswap` and `swapon`)

Output

(No output on success)

Notes

- Not all filesystems support `fallocate`. It is well-supported on `ext4`, `xfs`, `btrfs`, and `ocfs2`.
- If the filesystem doesn't support `fallocate`, you can use `'truncate -s SIZE FILENAME'` as a fallback, though it creates a sparse file.
- For older systems or filesystems, `'dd if=/dev/zero of=FILENAME bs=1M count=SIZE_IN_MB'` is the most compatible but much slower method.

Warnings

- This command will overwrite the specified file if it already exists.
- On some filesystems, the allocated space may contain 'stale' data from previously deleted files, though modern Linux filesystems (`ext4`, `xfs`) zero-fill or mark it as uninitialized for security.
- Ensure you have enough free disk space before running this command.

See Also

- `find-large-files-recursive`
- `disk-space-usage-per-directory`

Status

reviewed

Safety

caution

Shell

bash

Platforms

linux

Created At

2026-04-21

Updated At

2026-04-21

3.4 Disk Space Sort Largest Directories

Sort directories by disk usage and display the largest ones

Author: Marcos de Carvalho

Date: 2026-04-21

Displays the top 10 directories with the largest disk usage in the current directory, sorted in descending order by size in human-readable format.

Language

bash

Category

disk-usage

Command

```
du -sh */ | sort -hr | head -10
```

Explanation

The command uses 'du -sh' to calculate the total disk usage for each directory in human-readable format. The '*/' glob pattern matches only directories (not files). The output is piped to 'sort -hr' to sort in descending order using human-readable number comparison, with the largest directories appearing first. Finally, 'head -10' limits output to the top 10 directories.

Tags

du, disk-usage, directories, storage, sort, system-administration

Dependencies

coreutils

Arguments

1. **PATH** (Optional): Directory path to analyze (default: current directory)
Default: .
2. **COUNT** (Optional): Number of largest directories to display (default: 10)
Default: 10

Examples

1. `du -sh */ | sort -hr | head -10` - Show top 10 largest subdirectories in current directory
2. `du -sh */ | sort -hr` - Show all subdirectories sorted by size without limit
3. `du -sh */ | sort -hr | head -5` - Show top 5 largest subdirectories
4. `du -sh ./ * | sort -hr | head -10` - Include hidden files and directories (those starting with dot)
5. `cd /var && du -sh */ | sort -hr | head -10` - Find largest subdirectories in /var

Output

```
847G    cache/
512G    data/
256G    backups/
128G    logs/
96G     documents/
64G     downloads/
32G     pictures/
16G     videos/
8.5G    archives/
2.1G    temp/
```

Notes

- The `*/` pattern matches only directories; use `*/` to include files as well
- The `-h` flag makes output human-readable (K, M, G, T); use `-B` for a specific block size
- The `-s` flag summarizes each directory's total size instead of listing contents recursively
- Hidden directories (starting with `.`) are excluded; use `./ *` or `*/` (with `shopt`) to include them
- On systems with many subdirectories, results are calculated quickly since `du` only traverses each directory once
- Symbolic links are not followed by default; add `-L` flag to follow symlinks
- Results may vary if some directories are mounted on different filesystems or are inaccessible due to permissions

Warnings

- Results may be incomplete or misleading if you lack read permissions on some directories
- On network filesystems (NFS, SMB), the command may be slower due to network latency
- If dealing with millions of directories, the command can consume considerable memory and time
- Sparse files may cause inaccurate size reporting on some filesystems

See Also

- `find-largest-storage-users`
- `find-large-files-recursive`

Status

reviewed

Safety

safe

Shell

bash

Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

Created At

2026-04-21

Updated At

2026-04-21

3.5 Disk Space Usage Per Directory

Display disk space usage for all directories with hierarchical breakdown

Author: marcos

Date: 2026-04-21

Shows disk space usage for directories up to 2 levels deep in the current directory tree, sorted by size in descending order with human-readable format, providing a hierarchical breakdown of disk consumption.

Language

bash

Category

disk-usage

Command

```
du -h --max-depth=2 . | sort -hr
```

Explanation

The command uses 'du -h' to calculate disk usage in human-readable format. The '-max-depth=2' flag limits output to directories up to 2 levels deep from the current directory, avoiding excessive detail while still showing the hierarchy. The '.' specifies the starting directory. The output is piped to 'sort -hr' to sort in descending order by human-readable sizes, making it easy to identify which directories consume the most space at each level.

Tags

du, disk-usage, directories, storage, hierarchical, sort, system-administration

Dependencies

coreutils

Arguments

1. **PATH** (Optional): Root directory to analyze (default: current directory)
Default: .
2. **DEPTH** (Optional): Maximum directory depth to traverse (default: 2)
Default: 2

Examples

1. `du -h --max-depth=2 . | sort -hr` - Show disk usage up to 2 levels deep from current directory
2. `du -h --max-depth=1 . | sort -hr` - Show only immediate subdirectories (1 level deep)
3. `du -h --max-depth=3 /home | sort -hr` - Show disk usage in /home up to 3 levels deep
4. `du -h --max-depth=2 . | sort -hr | head -20` - Show top 20 directories by size (2 levels deep)
5. `du -ah --max-depth=2 . | sort -hr | head -15` - Include files in the listing, show top 15 items

Output

```
1.2T      .
847G      ./data
512G      ./data/cache
256G      ./data/backups
244G      ./media
128G      ./media/videos
116G      ./media/archives
156G      ./documents
96G       ./documents/projects
60G       ./documents/reports
```

Notes

- The `-max-depth` flag controls recursion depth: 0 shows only the starting directory, 1 shows immediate children, etc.
- The `-h` flag makes output human-readable (K, M, G, T); use `-B 1M` for sizes in fixed block units
- The `-s` flag is not needed here as `du` summarizes by default
- Hidden directories (those starting with `.`) are included automatically
- Each line shows cumulative size for that directory and all its contents
- Symbolic links are not followed by default; add `-L` to follow symlinks
- On filesystems with many nested directories, increasing `-max-depth` may significantly increase runtime

Warnings

- Results may be incomplete if you lack read permissions on some directories
- On network filesystems (NFS, SMB), this command can be slow due to network latency
- Very deep directory hierarchies combined with high `-max-depth` values may cause excessive output
- Sparse files may report misleading sizes on some filesystems

See Also

- `disk-space-sort-largest-directories`

Status

reviewed

Safety

safe

Shell

bash

Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

Created At

2026-04-21

Updated At

2026-04-21

3.6 Dmesg Errors Pretty

Kernel errors/warnings summary with full severity levels and colored output

Author: Marcos de Carvalho

Date: 2026-04-21

Retrieves all kernel messages from the kernel ring buffer, extracts and counts all message severity levels (emerg, alert, crit, err, warn, notice, info, debug), and presents them with a comprehensive summary front-matter followed by the human-readable colorized logs.

Language

bash

Category

diagnostics

Command

```
dmesg -T -x --level=emerg,alert,crit,err,warn --color=always 2>/dev/null | awk '
BEGIN { print "\033[1m=== KERNEL MESSAGE SUMMARY ===\033[0m" } { lines[NR] = \
$0; lvl = \ $2; gsub(/\x1B\[ [0-9;]*[mK]/, "", lvl); gsub(/[: \t]/, "", lvl);
if(lvl ~ /^(emerg|alert|crit|err|warn|notice|info|debug)\$/ ) cnt[lvl]++; if(
lvl ~ /^w+\$/ ) all_cnt[lvl]++ } END { if(NR>0) { print "\033[1m--- ERRORS &
WARNINGS ---\033[0m"; for(l in cnt) printf "\t %-7s : %d\n", l, cnt[l]; print
"\033[1m--- ALL MESSAGES (by severity) ---\033[0m"; for(l in all_cnt) printf "\t
%-7s : %d\n", l, all_cnt[l]; print "\033[1m
=====\033[0m\n"; for(i=1; i<=NR; i++) print
lines[i] } else print "No kernel messages found for selected levels." }'
```

Explanation

The one-liner uses 'dmesg -T -x' to extract kernel messages with human-readable timestamps (-T) and decode the facility/level prefixes (-x). We filter for all severity levels (emerg,alert,crit,err,warn,notice,info,debug) via '-level'. The '-color=always' flag preserves standard color coding. The output is piped into 'awk', which does several tasks: (1) strips ANSI escape sequences from the second column to cleanly count occurrences, (2) maintains two counters: one for 'errors/warnings' (emerg,alert,crit,err,warn) and another for 'all messages' (all severity levels), and (3) stores all log lines in memory. The END block prints a formatted summary frontmatter with both the 'errors/warnings' and 'all messages' counts, followed by the original colorized logs.

Tags

dmesg, kernel, errors, warnings, logs, troubleshooting, monitoring, summary, awk, diagnostics, severity, log-analysis, linux, system, administration

Dependencies

util-linux, gawk

Arguments

None

Examples

1. `dmesg -T -x --level=emerg,alert,crit,err,warn --color=always 2>/dev/null | awk 'BEGIN { print "\033[1m=== KERNEL MESSAGE SUMMARY ===\033[0m" } { lines[NR] = \033[0; lvl = \033[2; gsub(/\033[x1B\{[0-9;]*[mK]/, \{"\{"", lvl); gsub(/[: \t]/, \{"\{"", lvl); if(lvl ~ /^(emerg|alert|crit|err|warn|notice|info|debug)\{$/) cnt[lvl]++; if(lvl ~ /\w+\{$/) all_cnt[lvl]++ } END { if(NR>0) { print "\033[1m--- ERRORS & WARNINGS ---\033[0m"; for(l in cnt) printf \{" %-7s : %d\{n\{"", l, cnt[l]; print "\033[1m--- ALL MESSAGES (by severity) ---\033[0m"; for(l in all_cnt) printf \{" %-7s : %d\{n\{"", l, all_cnt[l]; print "\033[1m=====\033[0m\{n"; for(i=1; i<=NR; i++) print lines[i] } else print "No kernel messages found for selected levels." }' - Show full severity summary and colorized kernel messages`
2. `dmesg -T -x --level=emerg,alert,crit,err,warn --color=always 2>/dev/null | awk 'BEGIN { print "\033[1m=== KERNEL MESSAGE SUMMARY ===\033[0m" } { lines[NR] = \033[0; lvl = \033[2; gsub(/\033[x1B\{[0-9;]*[mK]/, \{"\{"", lvl); gsub(/[: \t]/, \{"\{"", lvl); if(lvl ~ /^(emerg|alert|crit|err|warn|notice|info|debug)\{$/) cnt[lvl]++; if(lvl ~ /\w+\{$/) all_cnt[lvl]++ } END { if(NR>0) { print "\033[1m--- ERRORS & WARNINGS ---\033[0m"; for(l in cnt) printf \{" %-7s : %d\{n\{"", l, cnt[l]; print "\033[1m--- ALL MESSAGES (by severity) ---\033[0m"; for(l in all_cnt) printf \{" %-7s : %d\{n\{"", l, all_cnt[l]; print "\033[1m=====\033[0m\{n"; for(i=1; i<=NR; i++) print lines[i] } else print "No kernel messages found for selected levels." }' | less -R - Show the summary and logs with a pager, preserving colors`
3. `sudo dmesg -T -x --level=emerg,alert,crit,err,warn --color=always 2>/dev/null | awk 'BEGIN { print "\033[1m=== KERNEL MESSAGE SUMMARY ===\033[0m" } { lines[NR] = \033[0; lvl = \033[2; gsub(/\033[x1B\{[0-9;]*[mK]/, \{"\{"", lvl); gsub(/[: \t]/, \{"\{"", lvl); if(lvl ~ /^(emerg|alert|crit|err|warn|notice|info|debug)\{$/) cnt[lvl]++; if(lvl ~ /\w+\{$/) all_cnt[lvl]++ } END { if(NR>0) { print "\033[1m--- ERRORS & WARNINGS ---\033[0m"; for(l in cnt) printf \{" %-7s : %d\{n\{"", l, cnt[l]; print "\033[1m--- ALL MESSAGES`

```
(by severity) ---\{}033[0m"; for(l in all_cnt) printf \{" %-7s : %d\{}\n\{}", l, all_cnt[l];
print "\{}033[1m=====\\{}033[0m\{}\n"; for(i=1; i<=NR; i++)
print lines[i] } else print "No kernel messages found for selected levels." }' - Run with
sudo to access kernel ring buffer on restricted systems
```

Output

```
\u001b[1m=== KERNEL MESSAGE SUMMARY ===\u001b[0m
\u001b[1m--- ERRORS & WARNINGS ---\u001b[0m
  err      : 2
  warn     : 3
  crit     : 1
\u001b[1m--- ALL MESSAGES (by severity) ---\u001b[0m
  err      : 2
  warn     : 3
  crit     : 1
  notice   : 5
  info     : 12
  debug    : 0
\u001b[1m=====\\u001b[0m

kern :err   : [Tue Apr 21 10:00:00 2026] sd 0:0:0:0: [sda] Write cache: enabled
kern :warn  : [Tue Apr 21 10:01:23 2026] WARNING: CPU: 0 PID: 1234
kern :crit  : [Tue Apr 21 10:02:45 2026] Critical hardware error
kern :warn  : [Tue Apr 21 10:05:00 2026] ext4-fs: mounted with warnings
kern :err   : [Tue Apr 21 10:06:12 2026] usb 1-1: device descriptor read/64,
      error -71
kern :warn  : [Tue Apr 21 10:08:34 2026] IPv6: ADDRCONF(NETDEV_UP): eth0: link is
      not ready
kern :notice: [Tue Apr 21 10:10:00 2026] Kernel log system initialized
kern :info  : [Tue Apr 21 10:15:00 2026] Network interface eth0 up and running
```

Notes

- Requires read access to the kernel ring buffer, which often requires sudo or being in the 'adm' group on some distributions.
- The '-x' (-decode) flag is critical here as it explicitly outputs the severity level on every line, making awk parsing reliable.
- ANSI stripping regex in awk ensures color codes don't interfere with severity counting.
- This command counts all message severity levels: emerg, alert, crit, err, warn, notice, info, debug.
- The 'errors & warnings' summary includes: emerg, alert, crit, err, warn.
- The 'all messages' summary includes all severity levels, providing complete log statistics.
- The entire dmesg output is stored in awk's memory (the 'lines' array) before printing, which is perfectly safe for typical volumes but could consume slightly more memory on massively flooded systems.

Warnings

- Running without sufficient privileges (like sudo) will result in a blank output or a permission denied error on most modern secure Linux distributions.

See Also

- `find-large-files-recursive`
- `disk-space-sort-largest-directories`

Status

reviewed

Safety

safe

Shell

bash

Platforms

linux

Created At

2026-04-21

Updated At

2026-04-21

3.7 Find 10 Largest Storage Users

Find users with largest storage usage

Author: Marcos de Carvalho

Date: 2026-04-21

Displays the top 10 users on the system by storage consumption in their home directories, sorted in descending order by disk usage.

Language

bash

Category

disk-usage

Command

```
du -sh /home/* 2>/dev/null | sort -hr | head -10
```

Explanation

The command uses 'du -sh' to calculate the total disk usage in human-readable format for each home directory. The '2>/dev/null' suppresses permission denied errors for inaccessible directories. The output is piped to 'sort -hr' to sort in descending order (largest first) using human-readable number comparison. Finally, 'head -10' limits the output to the top 10 users.

Tags

disk-usage, storage, users, system-administration, du, sort

Dependencies

coreutils, findutils

Arguments

1. **PATH** (Optional): Home directory path to analyze (default: /home)
Default: /home
2. **COUNT** (Optional): Number of top users to display (default: 10)
Default: 10

Examples

1. `du -sh /home/* 2>/dev/null | sort -hr | head -10` - Show top 10 users by storage usage in /home
2. `du -sh /home/* 2>/dev/null | sort -hr` - Show all users sorted by storage usage without limit
3. `du -sh /home/* 2>/dev/null | sort -hr | head -5` - Show top 5 users by storage usage
4. `du -sh /root /home/* 2>/dev/null | sort -hr | head -10` - Include root's home directory in the analysis

Output

```
450G    /home/alice
320G    /home/bob
215G    /home/charlie
156G    /home/diana
98G     /home/eve
67G     /home/frank
45G     /home/grace
32G     /home/henry
28G     /home/iris
19G     /home/jack
```

Notes

- The command suppresses permission errors with `'2_i/dev/null'`, allowing users to see results without elevated privileges
- Human-readable format (`-h`) displays sizes in K, M, G, etc., making interpretation easier
- Sort with `-h` flag properly handles human-readable numbers (not lexicographic sorting)
- Symbolic links in `/home` are counted separately if they point outside `/home`
- Hidden files and directories within user homes are included in the totals
- Results may vary if some home directories are mounted on different filesystems

Warnings

- Results may be incomplete if you lack read permissions on certain home directories
- On systems with many users, this command may take considerable time to complete
- NFS-mounted or slow storage may cause noticeable delays

See Also

- `find-large-files-recursive`
- `disk-space-usage-per-directory`

Status

reviewed

Safety

safe

Shell

bash

Platforms

linux, gnu-linux, freebsd

Created At

2026-04-21

Updated At

2026-04-21

3.8 Find Large Files Recursive

Find large files recursively in a directory tree

Author: marcos

Date: 2026-04-21

Recursively finds all regular files larger than 100 MB in the directory tree, displays them with human-readable sizes, and sorts by size in descending order.

Language

bash

Category

filesystem

Command

```
find . -type f -size +100M -exec ls -lh {} + | awk '{print $5, $9}' | sort -hr
```

Explanation

The command uses 'find' to recursively traverse the directory tree, filter for regular files (-type f), and select those exceeding 100 MB (-size +100M). The '-exec ls -lh {} +' displays detailed information for each file in a batch operation, which is more efficient than using '\{\};;'. The output is piped to 'awk' to extract the file size and path, then 'sort -hr' sorts the results by human-readable sizes in descending order, showing the largest files first.

Tags

find, filesystem, disk-usage, large-files, recursive, storage

Dependencies

findutils, coreutils

Arguments

1. **PATH** (Optional): Root directory to start the recursive search (default: current directory)
Default: .
2. **SIZE** (Optional): Minimum file size threshold (default: 100M, supports K, M, G suffixes)
Default: 100M

Examples

1. `find . -type f -size +100M -exec ls -lh {} + | awk '{print $5, $9}' | sort -hr` - Find files larger than 100 MB in current directory and subdirectories
2. `find /var -type f -size +500M -exec ls -lh {} + | awk '{print $5, $9}' | sort -hr` - Find large files in /var directory
3. `find . -type f -size +1G -exec ls -lh {} + | awk '{print $5, $9}' | sort -hr` - Find files larger than 1 GB
4. `find . -type f -size +50M -exec ls -lh {} + | awk '{print $5, $9}' | sort -hr | head -20` - Show top 20 largest files over 50 MB

Output

```
1.5G ./videos/archive.tar.gz
987M ./backups/database.sql.bz2
756M ./datasets/training-data.bin
543M ./cache/node_modules.tar
387M ./downloads/ubuntu-22.04-iso
256M ./logs/app-2026.log
198M ./tmp/large-temp-file
```

Notes

- The `-size` option supports various units: `c` (bytes), `k` (kilobytes), `M` (megabytes), `G` (gigabytes), and `b` (512-byte blocks)
- By default, `-size` rounds up (e.g., `+100M` matches files larger than 100 MiB)
- Use `-size -100M` to find files smaller than 100 MB
- The command may take considerable time on large directory trees
- Using `'{} +'` instead of `'{} \{'` is more efficient as it batches the command executions
- Results may be incomplete in directories where you lack read permissions

Warnings

- On network filesystems (NFS, SMB), this command may be significantly slower
- Does not follow symbolic links by default; add `-L` flag if needed
- Very large directory trees may cause the command to run for extended periods
- Some filesystems may not report accurate sizes for sparse files

See Also

- `find-largest-storage-users`
- `disk-space-sort-largest-directories`

Status

reviewed

Safety

safe

Shell

bash

Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

Created At

2026-04-21

Updated At

2026-04-21

3.9 Journalctl Errors Today

Show system executables with errors/failures from today's journal

Author: marcos

Date: 2026-04-22

Lists system executables that have generated failure, error, or fatal messages in today's journal logs, sorted by frequency.

Language

bash

Category

system-monitoring

Command

```
echo "=== Errors/Failures in Today's Journal ==="; journalctl --no-pager --since
today --grep 'fail|error|fatal' --output json | jq -r '._EXE' | sort | uniq -c
| sort -nr | awk '{printf "%4d %s\n", $1, $2}'
```

Explanation

The command retrieves all journal entries from today (since midnight), filters for messages containing the keywords 'fail', 'error', or 'fatal' (case-insensitive regex), outputs the results in JSON format for parsing, extracts the executable name (`_EXE` field) from each log entry, counts occurrences of each executable, sorts them numerically in descending order (most frequent first), and formats the output with a header and aligned columns for readability.

Tags

journalctl, systemd, logs, monitoring, troubleshooting, errors, failure, system-administration, diagnostics, debugging, logging, bash, jq, text-processing, process-management

Dependencies

systemd, jq

Arguments

None

Examples

1. `journalctl-errors-today` - Show errors/failures from today's journal
2. `journalctl --no-pager --since yesterday --grep 'fail|error|fatal' --output json | jq -r '._EXE' | sort | uniq -c | sort -nr` - Show errors/failures from yesterday's journal (raw output)

Output

```
=== Errors/Failures in Today's Journal ===
 5  /usr/bin/systemd
 3  /usr/bin/foo-daemon
 1  /usr/lib/bar-service
```

Notes

- Requires `journalctl` (`systemd`) and `jq` to be installed
- Only shows entries from the current day (midnight to now)
- Requires appropriate permissions to read journal logs
- The regex pattern 'fail—error—fatal' matches any of these words in log messages
- Output is sorted by count (highest first) for easy identification of frequent issues

Warnings

None

See Also

- dmesg-errors-pretty
- user-activity-and-quota-report

Status

reviewed

Safety

safe

Shell

bash

Platforms

linux, gnu-linux

Created At

2026-04-22

Updated At

2026-04-22

3.10 List All Aliases

List all shell aliases in a colored, formatted display

Author: marcos

Date: 2026-04-25

Lists all aliases defined in the user's bash shell configuration file in alphabetical order with colored, formatted output. Works with bash, zsh, and other shells on Linux and BSD systems.

Language

bash

Category

shell-utilities

Command

```
bash -c 'shell_conf="${1:-$HOME/.bashrc}"; [ -f "$shell_conf" ] || { echo "
Configuration file not found: $shell_conf"; exit 1; }; grep -E "^[[:space:]]*
alias[[:space:]]" "$shell_conf" | sed -E "s/^[[:space:]]*alias[[:space:]]*/" |
sort -f | while IFS= read -r line; do alias_name=${line%%=*}; alias_cmd=${line
#*=}; printf "\033[1;34m%-20s\033[0m -> \033[1;32m%s\033[0m\n" "
$alias_name" "$alias_cmd"; done' _
```

Explanation

This command reads the user's shell configuration file from the path passed as the first argument (defaulting to `~/.bashrc`), extracts alias definitions, sorts them alphabetically by alias name, and prints the alias name in blue and the stored alias command in green. It preserves the command text as written after the equals sign, which keeps the direct shell syntax intact and avoids fragile quote-stripping logic.

Tags

shell, alias, utilities, configuration, listing

Dependencies

bash, grep, sed, sort

Arguments

1. **CONFIG_FILE** (Optional): Path to shell configuration file (optional, defaults to `~/.bashrc`)
Default: `~/.bashrc`

Examples

1. `bash -c 'shell_conf="${1:-$HOME/.bashrc}"; [-f "$shell_conf"] || { echo "Configuration file not found: $shell_conf"; exit 1; }; grep -E "^[[:space:]]*alias[[:space:]]" "$shell_conf" | sed -E "s/^[[:space:]]*alias[[:space:]]*/" | sort -f | while IFS= read -r line; do alias_name=${line%%=*}; alias_cmd=${line#*=}; printf "\033[1;34m%-20s\033[0m -> \033[1;32m%s\033[0m\n" "$alias_name" "$alias_cmd"; done' _` - List all aliases from the default shell configuration file in alphabetical order
2. `bash -c 'shell_conf="${1:-$HOME/.bashrc}"; [-f "$shell_conf"] || { echo "Configuration file not found: $shell_conf"; exit 1; }; grep -E "^[[:space:]]*alias[[:space:]]" "$shell_conf" | sed -E "s/^[[:space:]]*alias[[:space:]]*/" | sort -f | while IFS= read -r line; do alias_name=${line%%=*}; alias_cmd=${line#*=}; printf "\033[1;34m%-20s\033[0m -> \033[1;32m%s\033[0m\n" "$alias_name" "$alias_cmd"; done' _` `~/.zshrc` - List all aliases from a zsh configuration file in alphabetical order
3. `alias list-aliases='bash -c '""'shell_conf="${1:-$HOME/.bashrc}"; [-f "$shell_conf"] || { echo "Configuration file not found: $shell_conf"; exit 1; }; grep -E "^[[:space:]]*alias[[:space:]]" "$shell_conf" | sed -E "s/^[[:space:]]*alias[[:space:]]*/" | sort -f | while IFS= read -r line; do alias_name=${line%%=*}; alias_cmd=${line#*=}; printf "\033[1;34m%-20s\033[0m -> \033[1;32m%s\033[0m\n" "$alias_name" "$alias_cmd"; done' ""' _` - Create a permanent alias for listing all aliases in alphabetical order

Output

```
\033[1;34mdocker-rm-stopped \033[0m -> \033[1;32m'docker rm $(docker ps -aq --
    filter status=exited)'\033[0m
\033[1;34mgs \033[0m -> \033[1;32m"git status"\033[0m
\033[1;34mll \033[0m -> \033[1;32m'ls -la'\033[0m
\033[1;34mup \033[0m -> \033[1;32m'uptime'\033[0m
```

Notes

- The command works with bash, zsh, and other shells by specifying the appropriate configuration file
- Color codes: alias names are displayed in blue, commands in green
- Alias names are listed in alphabetical order
- The command text is shown exactly as stored after the equals sign
- Can be used directly or converted to a permanent alias
- Works on Linux and BSD systems

Warnings

- Only shows aliases defined in the specified configuration file
- Does not display aliases defined in sessions or other files
- May not work with non-standard alias definitions

See Also

- `replace-string-in-files`

Status

reviewed

Safety

safe

Shell

bash

Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

Created At

2026-04-25

Updated At

2026-04-25

3.11 Nonhuman User Activity And Quota Report

Comprehensive system/service non-human user report with disk quota, usage, and activity

Author: Marcos de Carvalho

Date: 2026-04-21

Generates a comprehensive system audit report for all non-human (system/service) users (requires sudo) showing username, disk quota limit, home directory path, actual disk usage, account creation date, file count, last login timestamp, and last executed command. All information is formatted in a readable table with elevated privileges for complete data access.

Language

bash

Category

system-administration

Command

```
sudo bash -c "printf 'USER\tQUOTA\tHOME\tDISK_USAGE\tCREATED\tFILES\tLAST_LOGIN\tLAST_CMD\n'; getent passwd | awk -F: ' \$3 < 1000 || \$1 ~ /^nobody$/ { print \$1, \$6 }' | while read u h; do q=\$(quota -u \"\$u\" 2>/dev/null | tail -1 | awk '{print \$2}'); [ -z \"\$q\" ] && q=none; d=\$(du -sh \"\$h\" 2>/dev/null | cut -f1); c=\$(stat -c %y \"\$h\" 2>/dev/null | cut -d' ' -f1 || echo N/A); f=\$(find \"\$h\" -type f 2>/dev/null | wc -l); l=\$(lastlog -u \"\$u\" 2>/dev/null | tail -1 | awk '{print \$5, \$6, \$7}'); [ -z \"\$l\" ] && l=Never; cmd=\$(tail -1 \"\$h\"/.bash_history 2>/dev/null | head -c 30); [ -z \"\$cmd\" ] && cmd=N/A; printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \"\$u\" \"\$q\" \"\$h\" \"\$d\" \"\$c\" \"\$f\" \"\$l\" \"\$cmd\"; done" | column -t -s $'\t'
```

Explanation

The command uses 'sudo bash -c' to execute the entire script with elevated privileges, ensuring access to all user data. It retrieves users with UID < 1000 (or 'nobody') via getent, then for each user gathers: disk quota from 'quota -u', home disk usage from 'du -sh', account creation date via 'stat', file count using 'find', last login from 'lastlog', and last command from .bash_history. The 'bash -c' wrapper allows proper variable expansion and quoting within the sudo context. Output is joined by tabs and formatted by 'column -t' for readability, avoiding issues with spaces in values.

Tags

user-management, disk-usage, system-administration, activity, report, quota, audit, system-users, service-accounts

Dependencies

coreutils, util-linux, shadow-utils, findutils

Arguments

1. **UID_THRESHOLD** (Optional): Maximum UID to consider a system user (default: 1000 for non-human users)
Default: 1000

Examples

1.

```
sudo bash -c "printf 'USER\tQUOTA\tHOME\tDISK_USAGE\tCREATED\tFILES\tLAST_LOGIN\tLAST_CMD\n'; getent passwd | awk -F: '{ if ($3 < 1000 || $1 ~ /^nobody$/ { print $1,$6 } }' | while read u h; do q=$(quota -u \"$u\" 2>/dev/null | tail -1 | awk '{print $2}'); [ -z \"$q\" ] && q=None; d=$(du -sh \"$h\" 2>/dev/null | cut -f1); c=$(stat -c %y \"$h\" 2>/dev/null | cut -d' ' -f1 || echo N/A); f=$(find \"$h\" -type f 2>/dev/null | wc -l); l=$(lastlog -u \"$u\" 2>/dev/null | tail -1 | awk '{print $5,$6,$7}'); [ -z \"$l\" ] && l=Never; cmd=$(tail -1 \"$h\"/.bash_history 2>/dev/null | head -c 30); [ -z \"$cmd\" ] && cmd=N/A; printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \"$u\" \"$q\" \"$h\" \"$d\" \"$c\" \"$f\" \"$l\" \"$cmd\"; done" | column -t -s $'\t' - Generate complete report for all non-human (system) users with all metrics (requires sudo)
```
2.

```
sudo bash -c "printf 'USER\tQUOTA\tDISK_USAGE\tCREATED\tFILES\tLAST_LOGIN\n'; getent passwd | awk -F: '{ if ($3 < 1000 || $1 ~ /^nobody$/ { print $1,$6 } }' | while read u h; do q=$(quota -u \"$u\" 2>/dev/null | tail -1 | awk '{print $2}'); [ -z \"$q\" ] && q=None; d=$(du -sh \"$h\" 2>/dev/null | cut -f1); c=$(stat -c %y \"$h\" 2>/dev/null | cut -d' ' -f1 || echo N/A); f=$(find \"$h\" -type f 2>/dev/null | wc -l); l=$(lastlog -u \"$u\" 2>/dev/null | tail -1 | awk '{print $5,$6,$7}'); [ -z \"$l\" ] && l=Never; printf '%s\t%s\t%s\t%s\t%s\t%s\n' \"$u\" \"$q\" \"$d\" \"$c\" \"$f\" \"$l\"; done" | sort -t $'\t' -k3 -hr" | column -t -s $'\t' - Sort by disk usage descending, exclude home path and last command
```
3.

```
sudo bash -c "printf 'USER\tQUOTA\tDISK_USAGE\tFILES\n'; getent passwd | awk -F: '{ if ($3 < 1000 || $1 ~ /^nobody$/ { print $1,$6 } }' | while read u h; do q=$(quota -u \"$u\" 2>/dev/null | tail -1 | awk '{print $2}'); [ -z \"$q\" ] && q=None; d=$(du -sh \"$h\" 2>/dev/null | cut -f1); f=$(find \"$h\" -type f 2>/dev/null | wc -l); printf '%s\t%s\t%s\t%s\n' \"$u\" \"$q\" \"$d\" \"$f\"; done" | column -t -s $'\t' - Simplified report showing only quota, disk usage, and file count
```

Output

USER	QUOTA	HOME	DISK_USAGE	CREATED	FILES	LAST_LOGIN	LAST_CMD
------	-------	------	------------	---------	-------	------------	----------

root	none	/root	1.5G	2023-01-15	15340	May 20 13:45	apt
update							
daemon	none	/usr/sbin	32K	2023-01-15	12	Never	N/A
bin	none	/bin	0	2023-01-15	0	Never	N/A
www-data	none	/var/www	4.2G	2023-05-10	45678	Never	N/A
nobody	none	/nonexistent	0	2023-01-15	0	Never	N/A

Notes

- This command is designed to run with sudo for complete information; running without sudo will show incomplete results
- The 'sudo bash -c' wrapper preserves the entire command context and allows proper variable expansion
- Disk quota requires quota system to be enabled on the filesystem; shows 'none' if unavailable
- File count calculation uses 'find' which can be slow on directories with millions of files
- Account creation date uses home directory birth time (%w) if available, falling back to modification time (%y)
- Last command is truncated to 30 characters; extracted from .bash_history. Note: Bash only flushes history to disk upon session exit or when 'history -a' is run, so this shows the last saved command, not necessarily the active session's most recent command.
- Users with UIDs $\neq$ 1000 are standard human accounts and are excluded by default. Note that system/service accounts typically lack valid login shells or history files, making LAST_CMD frequently N/A and LAST_LOGIN Never.
- The command uses getent instead of /etc/passwd for better compatibility with different user databases (LDAP, NIS, etc.)

Warnings

- Requires sudo access to function properly; user must have sudo privileges without password prompt configured in sudoers for unattended execution
- Large file counts or extensive directory trees will cause significant performance degradation
- Running with find on large home directories can consume considerable IO and CPU resources
- The lslogins command retrieves last login information depending on wtmp/btmp logs; accuracy varies by distribution logging policies
- Last command information depends on .bash_history being available and enabled; other shells use different history files
- Network-mounted home directories (NFS, SMB) will significantly increase command execution time
- On systems with many users, overall execution time can be substantial; consider running during off-peak hours

See Also

- `find-largest-storage-users`
- `du-sort-largest-directories`
- `disk-space-usage-per-directory`
- `find-large-files-recursive`

Status

reviewed

Safety

caution

Shell

bash

Platforms

linux, gnu-linux

Created At

2026-04-21

Updated At

2026-04-21

3.12 Pretty Print Public Ssh Pgp Keys

Pretty-print SSH and PGP public keys with details

Author: Marcos de Carvalho

Date: 2026-05-03

Reads SSH public key files and a GnuPG public keyring, then prints paths, SSH key text, PGP identities, long key IDs, fingerprints, and armored public key blocks in a formatted report.

Language

bash

Category

cryptography

Command

```
bash -c 'sshdir=${1:-$HOME/.ssh}; gpghome=${2:-$HOME/.gnupg}; printf "\033[1m===
SSH PUBLIC KEYS (%s) ===\033[0m\n" "$sshdir"; mapfile -d "" s <<(find "$sshdir
" -type f \( -name "*.pub" -o -name authorized_keys \) -print0 2>/dev/null); ((
${#s[@]}) || echo "No SSH public key files found."; for f in "${s[@]}"; do
printf "\n\033[1;34m%s\033[0m\n" "$f"; ssh-keygen -lf "$f" 2>/dev/null | sed "s
/^/ details: /"; sed "s/^/ key: /" "$f"; done; printf "\n\033[1m=== PGP
PUBLIC KEYS (%s) ===\033[0m\n" "$gpghome"; if [ -e "$gpghome/pubring.kbx" ] ||
[ -e "$gpghome/pubring.gpg" ]; then GNUPGHOME="$gpghome" gpg --batch --list-
public-keys --with-fingerprint --with-subkey-fingerprint --keyid-format long
2>/dev/null | sed "s/^/ /"; printf "\n\033[1m=== ARMORED PGP PUBLIC KEY BLOCKS
===\033[0m\n"; GNUPGHOME="$gpghome" gpg --batch --armor --export 2>/dev/null;
else echo "No GnuPG public keyring found."; fi' _
```

Explanation

The command uses a `bash -c` wrapper so both optional paths are supplied as trailing arguments. `SSH_DIR` defaults to `~/ssh` and is searched recursively for `*.pub` files and `authorized_keys` files. Each matching SSH file is labeled, summarized with `ssh-keygen -lf`, and printed with indentation. `GNUPG_HOME` defaults to `~/gnupg`; when it contains `pubring.kbx` or `pubring.gpg`, GnuPG lists public keys with long key IDs, primary fingerprints, and subkey fingerprints, then exports all public keys as ASCII-armored blocks. The `_` placeholder acts as `$0` inside `bash -c` so `$1` and `$2` map cleanly to `SSH_DIR` and `GNUPG_HOME`.

Tags

ssh, pgp, gpg, public-keys, fingerprints, cryptography, audit, pretty-print

Dependencies

bash, find, ssh-keygen, sed, gpg

Arguments

1. **SSH_DIR** (Optional): Directory to search recursively for SSH public key files. Defaults to `~/ssh`.
Default: `~/ssh`
2. **GNUPG_HOME** (Optional): GnuPG home directory whose public keyring will be listed and exported. Defaults to `~/gnupg`.
Default: `~/gnupg`

Examples

1. `bash -c 'sshdir=${1:-$HOME/.ssh}; gpghome=${2:-$HOME/.gnupg}; printf "\033[1m=== SSH PUBLIC KEYS (%s) ===\033[0m\n" "$sshdir"; mapfile -d "" s <<(find "$sshdir" -type f \{ \} (-name "*.pub" -o -name authorized_keys \{ \}) -print0 2>/dev/null); ((${#s[@]}) || echo "No SSH public key files found."; for f in "${s[@]}"; do printf "\n\033[1;34m%s\033[0m\n" "$f"; ssh-keygen -lf "$f" 2>/dev/null | sed "s/^/ details: /"; sed "s/^/ key: /" "$f"; done; printf "\n\033[1m=== PGP PUBLIC KEYS (%s) ===\033[0m\n" "$gpghome"; if [-e "$gpghome/pubring.kbx"] || [-e "$gpghome/pubring.gpg"]; then GNUPGHOME="$gpghome" gpg --batch --list-public-keys --with-fingerprint --with-subkey-fingerprint --keyid-format`

- ```

long 2>/dev/null | sed "s/^/ /"; printf "\n\{}033[1m=== ARMORED PGP PUBLIC KEY BLOCKS
===\{}033[0m\{}\n"; GNUPGHOME="$gpghome" gpg --batch --armor --export 2>/dev/null; else
echo "No GnuPG public keyring found."; fi' - - Print SSH public keys from ~/.ssh and PGP
public keys from ~/.gnupg

2. bash -c 'sshdir=${1:-$HOME/.ssh}; gpghome=${2:-$HOME/.gnupg}; printf "\{}033[1m=== SSH
PUBLIC KEYS (%s) ===\{}033[0m\{}\n" "$sshdir"; mapfile -d "" s <<(find "$sshdir" -type
f \{}(-name "*.pub" -o -name authorized_keys \{})) -print0 2>/dev/null); ((${#s[@]})) ||
echo "No SSH public key files found."; for f in "${s[@]}"; do printf "\n\{}033[1;34m%s\{}033[0m\{}
"$f"; ssh-keygen -lf "$f" 2>/dev/null | sed "s/^/ details: /"; sed "s/^/ key: /" "$f";
done; printf "\n\{}033[1m=== PGP PUBLIC KEYS (%s) ===\{}033[0m\{}\n" "$gpghome"; if [
-e "$gpghome/pubring.kbx"] || [-e "$gpghome/pubring.gpg"]; then GNUPGHOME="$gpghome"
gpg --batch --list-public-keys --with-fingerprint --with-subkey-fingerprint --keyid-format
long 2>/dev/null | sed "s/^/ /"; printf "\n\{}033[1m=== ARMORED PGP PUBLIC KEY BLOCKS
===\{}033[0m\{}\n"; GNUPGHOME="$gpghome" gpg --batch --armor --export 2>/dev/null; else
echo "No GnuPG public keyring found."; fi' - /etc/ssh ~/.gnupg - Print system SSH host pub-
lic keys from /etc/ssh and the current user's GnuPG public keyring

```

## Output

```

\u001b[1m=== SSH PUBLIC KEYS (/home/user/.ssh) ===\u001b[0m

\u001b[1;34m/home/user/.ssh/id_ed25519.pub\u001b[0m
 details: 256 SHA256:examplefingerprint user@example (ED25519)
 key: ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIExampleKey user@example

\u001b[1m=== PGP PUBLIC KEYS (/home/user/.gnupg) ===\u001b[0m
pub ed25519/0123456789ABCDEF 2026-05-03 [SC]
 0123 4567 89AB CDEF 0123 4567 0123 4567 89AB CDEF
uid [ultimate] User Example <user@example>
sub cv25519/FEDCBA9876543210 2026-05-03 [E]
 FEDC BA98 7654 3210 FEDC BA98 7654 3210 FEDC BA98

\u001b[1m=== ARMORED PGP PUBLIC KEY BLOCKS ===\u001b[0m
-----BEGIN PGP PUBLIC KEY BLOCK-----
...
-----END PGP PUBLIC KEY BLOCK-----

```

## Notes

- The SSH section includes \*.pub files and authorized\_keys files under SSH\_DIR.
- ssh-keygen -lf prints one fingerprint line per public key when a file contains multiple keys.
- The PGP section uses the selected GnuPG home directory and requires pubring.kbx or pubring.gpg to exist.
- Use a pager such as less -R when the exported PGP public key blocks are long.
- This command is intended for Bash because it uses mapfile and process substitution.

## Warnings

- Public keys are not secret, but they often include names, email addresses, hostnames, or comments that may be sensitive in shared logs or screenshots.

- The command does not print private SSH or PGP key material.
- GnuPG may create or refresh local keyring metadata such as trustdb.gpg when inspecting a GnuPG home that lacks it.

## See Also

- bulk-gpg-detach-sign-files

## Status

reviewed

## Safety

caution

## Shell

bash

## Platforms

linux, gnu-linux

## Created At

2026-05-03

## Updated At

2026-05-03

## 3.13 Replace String In Files

### Replace string in multiple files using sed

**Author:** Marcos de Carvalho

**Date:** 2026-04-21

Replaces all occurrences of a string with a replacement string in multiple files or file patterns. Pure oneliner that works directly pasted with parameters, or in an alias.

## Language

bash

## Category

text-processing

## Command

```
bash -c 'old=$1; new=$2; shift 2; find "$@" -type f -exec sed -i "s#$old#$new#g" {} +' _
```

## Explanation

This oneliner uses a subshell ('bash -c') to parse positional parameters. The first argument is the string to replace ('\$1'), the second is the replacement ('\$2'), and 'shift 2' leaves the remaining arguments as the files or patterns to search in ('\$@'). The 'find' command locates all matching files and processes them with 'sed' using the '-exec {} +' syntax, which efficiently handles multiple files in batches. The '\_' at the end acts as '\$0' (the script name) so '\$1' maps to the first actual argument. This approach handles an arbitrary amount of files and supports wildcard file patterns without hardcoding values into the command itself.

## Tags

sed, text-processing, string-replacement, file-editing, search-replace, batch-replace

## Dependencies

sed, coreutils

## Arguments

1. **SEARCH** (Required): The string to search for (first parameter passed after the command).
2. **REPLACE** (Required): The replacement string (second parameter passed after the command).
3. **FILES** (Required): One or more file paths or patterns (e.g., file1.txt, \*.txt, /path/\*.js).

## Examples

1. `bash -c 'old=$1; new=$2; shift 2; find "$@" -type f -exec sed -i "s#$old#$new#g" {} +' _ 'old-domain.com' 'new-domain.com' config.json settings.json` - Paste directly: replace 'old-domain.com' with 'new-domain.com' in config.json and settings.json
2. `bash -c 'old=$1; new=$2; shift 2; find "$@" -type f -exec sed -i "s#$old#$new#g" {} +' _ '/var/old/path' '/var/new/path' nginx.conf` - Replace a path containing slashes in a configuration file
3. `bash -c 'old=$1; new=$2; shift 2; find "$@" -type f -exec sed -i "s#$old#$new#g" {} +' _ 'old-text' 'new-text' *.txt` - Replace in all .txt files in current directory using a wildcard
4. `alias repl="bash -c 'old=\{}$1; new=\{}$2; shift 2; find \{}"\{}$@\{}" -type f -exec sed -i \{}"s#\{}$old#\{}$new#g\{}" {} +' _"` - Create an alias named 'repl' to easily perform string replacements
5. `repl 'deprecated_method' 'new_method' src/**/*.py` - Use the alias to replace in all Python files in src directory and subdirectories
6. `repl 'old-api' 'new-api' config/*.json docs/*.md` - Use the alias to replace in multiple file types across different directories

## Output

(No output on success - files are modified in place)

## Notes

- Pure oneliner: uses `bash -c` to accept arbitrary parameters after the command string
- Usage: Paste the command and append your search string, replacement string, and file patterns
- Can be easily turned into a powerful alias in your `.bashrc` or `.bash_profile`
- The `-i` flag modifies files in place; make backups first with `cp file.txt file.txt.bak` if concerned
- Using `#` as delimiter prevents issues when search/replace strings contain forward slashes
- The `g` flag replaces all occurrences per line; remove it to replace only first occurrence
- The `-exec {} +` syntax bundles multiple files to `sed` for efficiency
- Files with spaces or special characters are handled correctly by `find`
- For case-insensitive replacement, modify the command to include the `i` flag: `sed -i "s##$old#$new#g"`

## Warnings

- The `-i` flag modifies files directly without confirmation - always test on copies first
- If search string is empty, `sed` will replace every character with the replacement string
- The command will silently fail to modify files if they lack write permissions
- Very large files or many files may cause performance issues
- Replacement is literal - regex special characters will be treated as patterns (not escaped)
- Be careful with wildcard patterns - they may match more files than intended
- The `find` command only processes regular files, not directories
- Special characters in search/replace strings must be properly quoted or escaped

## See Also

- `find-large-files-recursive`
- `disk-space-usage-per-directory`

## Status

reviewed

## Safety

caution

## Shell

bash

## Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

## Created At

2026-04-21

## Updated At

2026-04-21

## 3.14 System Memory Report Top10

### System memory report with top 10 consuming processes

**Author:** Marcos de Carvalho

**Date:** 2026-04-22

Displays system memory usage in human-readable format and lists the top 10 memory-consuming processes by percentage of memory used and resident set size in MB.

## Language

bash

## Category

monitoring

## Command

```
echo -e "Memory Report:\n$(free -h)\n\nTop 10 Memory Consuming Processes (%MEM and\nRSS in MB):\n$(ps -eo pid,comm,%mem,rss --sort=-%mem | awk 'NR==1 {print $0 "\nRSS(MB)"}; NR>1 {printf "%s %s %s %.2f\n", $1, $2, $3, $4/1024}' | head -n 11)"
```

## Explanation

The command uses 'free -h' to show memory statistics in human-readable units (e.g., MB, GB). It then uses 'ps' to list all processes sorted by memory usage (%MEM) in descending order, showing the top 10 processes including their RSS (Resident Set Size) converted to MB. The awk command formats the output to display PID, COMMAND, %MEM, and RSS(MB) with proper headers. The output is formatted with clear section headers and spacing for readability.

## Tags

memory, system-monitoring, top-processes, ram, ps, free

## Dependencies

bash, coreutils, procp-ng

## Arguments

None

## Examples

1. `echo -e "Memory Report:\n$(free -h)\n\nTop 10 Memory Consuming Processes (%MEM and RSS in MB):\n$(ps -eo pid,comm,%mem,rss --sort=-%mem | awk 'NR==1 {print $0 " RSS(MB)"}; NR>1 {printf "%s %s %s %.2f\n", $1, $2, $3, $4/1024}' | head -n 11)"` - Run the memory report oneliner directly in the shell
2. `alias memreport='echo -e "Memory Report:\n$(free -h)\n\nTop 10 Memory Consuming Processes (%MEM and RSS in MB):\n$(ps -eo pid,comm,%mem,rss --sort=-%mem | awk 'NR==1 {print $0 " RSS(MB)"}; NR>1 {printf "%s %s %s %.2f\n", $1, $2, $3, $4/1024}' | head -n 11)"' && memreport` - Define as an alias and then execute it

## Output

|                                                         |             |      |          |        |            |           |
|---------------------------------------------------------|-------------|------|----------|--------|------------|-----------|
| Memory Report:                                          |             |      |          |        |            |           |
|                                                         | total       | used | free     | shared | buff/cache | available |
| Mem:                                                    | 7.7G        | 2.3G | 3.1G     | 256M   | 2.3G       | 5.0G      |
| Swap:                                                   | 2.0G        | 0B   | 2.0G     |        |            |           |
| Top 10 Memory Consuming Processes (%MEM and RSS in MB): |             |      |          |        |            |           |
| PID                                                     | COMMAND     | %MEM | RSS (MB) |        |            |           |
| 1234                                                    | firefox     | 15.2 | 1175.04  |        |            |           |
| 5678                                                    | chrome      | 12.8 | 991.42   |        |            |           |
| 9012                                                    | code        | 8.5  | 657.92   |        |            |           |
| 3456                                                    | thunderbird | 6.2  | 479.74   |        |            |           |
| 7890                                                    | slack       | 4.1  | 317.12   |        |            |           |
| 1122                                                    | python3     | 3.8  | 293.76   |        |            |           |
| 3344                                                    | java        | 3.5  | 270.85   |        |            |           |
| 5566                                                    | docker      | 2.9  | 224.26   |        |            |           |
| 7788                                                    | Xorg        | 2.5  | 193.28   |        |            |           |
| 9900                                                    | gnome-shell | 2.2  | 170.24   |        |            |           |

## Notes

- The %MEM column shows the percentage of available physical memory used by the process.
- The RSS(MB) column shows the Resident Set Size in megabytes (non-swapped physical memory used).
- The header line is included in the top 10 output for clarity.
- The awk command converts RSS from KB to MB by dividing by 1024 and formats to 2 decimal places.

## Warnings

None

## See Also

- `disk-usage-summary`
- `cpu-top-processes`

## Status

reviewed

## Safety

safe

## Shell

bash

## Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

## Created At

2026-04-22

## Updated At

2026-04-22

## 3.15 User Activity And Quota Report

### Comprehensive user system report with disk quota, usage, and activity

**Author:** Marcos de Carvalho

**Date:** 2026-04-21

Generates a comprehensive system audit report for all human users (requires sudo) showing username, disk quota limit, home directory path, actual disk usage, account creation date, file count, last login timestamp, and last executed command. All information is formatted in a readable table with elevated privileges for complete data access.

## Language

bash

## Category

system-administration

## Command

```
sudo bash -c "printf 'USER\tQUOTA\tHOME\tDISK_USAGE\tCREATED\tFILES\tLAST_LOGIN\tLAST_CMD\n'; getent passwd | awk -F: '\$3 >= 1000 { if (\$1 ~ /^nobody$/) next; print \$1,\$6 }' | while read u h; do q=\$(quota -u \"\$u\" 2>/dev/null | tail -1 | awk '{print \$2}'); [-z \"\$q\"] && q=none; d=\$(du -sh \"\$h\" 2>/dev/null | cut -f1); c=\$(stat -c %y \"\$h\" 2>/dev/null | cut -d' ' -f1 || echo N/A); f=\$(find \"\$h\" -type f 2>/dev/null | wc -l); l=\$(lastlog -u \"\$u\" 2>/dev/null | tail -1 | awk '{print \$5,\$6,\$7}'); [-z \"\$l\"] && l=Never; cmd=\$(tail -1 \"\$h\"/.bash_history 2>/dev/null | head -c 30); [-z \"\$cmd\"] && cmd=N/A; printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \"\$u\" \"\$q\" \"\$h\" \"\$d\" \"\$c\" \"\$f\" \"\$l\" \"\$cmd\"; done" | column -t -s $'\t'
```

## Explanation

The command uses 'sudo bash -c' to execute the entire script with elevated privileges, ensuring access to all user data. It retrieves users with UID  $\geq 1000$  via `getent`, then for each user gathers: disk quota from 'quota -u', home disk usage from 'du -sh', account creation date via 'stat', file count using 'find', last login from 'lastlog', and last command from `.bash_history`. The 'bash -c' wrapper allows proper variable expansion and quoting within the sudo context. Output is joined by tabs and formatted by 'column -t' for readability, avoiding issues with spaces in values.

## Tags

user-management, disk-usage, system-administration, activity, report, quota, audit, users

## Dependencies

coreutils, util-linux, shadow-utils, findutils

## Arguments

1. **UID\_THRESHOLD** (Optional): Minimum UID to consider a user (default: 1000 for human users only)  
Default: 1000

## Examples

1. `sudo bash -c "printf 'USER\tQUOTA\tHOME\tDISK_USAGE\tCREATED\tFILES\tLAST_LOGIN\tLAST_CMD\n'; getent passwd | awk -F: '\$3 >= 1000 { if (\$1 ~ /^nobody$/) next; print \$1,\$6 }' | while read u h; do q=\$(quota -u \"\$u\" 2>/dev/null | tail -1 | awk '{print \$2}'); [ -z \"\$q\" ] && q=none; d=\$(du -sh \"\$h\" 2>/dev/null | cut -f1); c=\$(stat -c %y \"\$h\" 2>/dev/null | cut -d' ' -f1 || echo N/A); f=\$(find \"\$h\" -type f 2>/dev/null | wc -l); l=\$(lastlog -u \"\$u\" 2>/dev/null | tail -1 | awk '{print \$5,\$6,\$7}'); [ -z \"\$l\" ] && l=Never; cmd=\$(tail -1 \"\$h\"/.bash_history 2>/dev/null | head -c 30); [ -z \"\$cmd\" ] && cmd=N/A;`

```
printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \{"\{"$u\{" \{"\{"$q\{" \{"\{"$h\{" \{"\{"$d\{" \{"\{"$c\{" \{"\{"$f\{" \{"\{"$l\{" \{"\{"$cmd\{""; done" | column
-t -s $'\{t' - Generate complete report for all human users with all metrics (requires sudo)
```

2. 

```
sudo bash -c "printf 'USER\tQUOTA\tDISK_USAGE\tCREATED\tFILES\tLAST_LOGIN\n';
getent passwd | awk -F: '{ if ($3 >= 1000 { if ($1 ~ /^nobody$/) next; print $1,$6
}}' | while read u h; do q=\{"\{"$(quota -u \{"\{"$u\{" 2>/dev/null | tail -1 | awk '{print
\{"\{"$2}'); [-z \{"\{"$q\{"] && q=none; d=\{"\{"$(du -sh \{"\{"$h\{" 2>/dev/null | cut
-f1); c=\{"\{"$(stat -c %y \{"\{"$h\{" 2>/dev/null | cut -d' ' -f1 || echo N/A); f=\{"\{"$(find
\{"\{"$h\{" -type f 2>/dev/null | wc -l); l=\{"\{"$(lastlog -u \{"\{"$u\{" 2>/dev/null
| tail -1 | awk '{print $5,$6,$7}'); [-z \{"\{"$l\{"] && l=Never; printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \{"\{"$u\{" \{"\{"$q\{" \{"\{"$d\{" \{"\{"$c\{" \{"\{"$f\{" \{"\{"$l\{""; done
| sort -t $'\{t' -k3 -hr" | column -t -s $'\{t' - Sort by disk usage descending, exclude
home path and last command
```
3. 

```
sudo bash -c "printf 'USER\tQUOTA\tDISK_USAGE\tFILES\n'; getent passwd | awk -F:
'\{"$3 >= 1000 { if ($1 ~ /^nobody$/) next; print $1,$6 }' | while read u h; do
q=\{"\{"$(quota -u \{"\{"$u\{" 2>/dev/null | tail -1 | awk '{print \{"$2}'); [-z \{"\{"$q\{"
] && q=none; d=\{"\{"$(du -sh \{"\{"$h\{" 2>/dev/null | cut -f1); f=\{"\{"$(find \{"\{"$h\{"
-type f 2>/dev/null | wc -l); printf '%s\t%s\t%s\t%s\n' \{"\{"$u\{" \{"\{"$q\{"
\{"\{"$d\{" \{"\{"$f\{""; done" | column -t -s $'\{t' - Simplified report showing only
quota, disk usage, and file count
```

## Output

| USER                  | QUOTA | HOME             | DISK_USAGE | CREATED    | FILES  | LAST_LOGIN   |
|-----------------------|-------|------------------|------------|------------|--------|--------------|
| LAST_CMD              |       |                  |            |            |        |              |
| marcos                | 5G    | /home/marcos     | 290G       | 2024-01-15 | 124356 | May 20 13:45 |
| grep -r pattern .     |       |                  |            |            |        |              |
| libvirt               | none  | /var/lib/libvirt | 16K        | 2023-06-01 | 3245   | Never        |
|                       |       | N/A              |            |            |        |              |
| test-user             | 2G    | /home/test-user  | 1.2G       | 2025-03-10 | 45678  | May 10 09:30 |
| ls -lah               |       |                  |            |            |        |              |
| admin                 | 10G   | /home/admin      | 3.5G       | 2024-11-22 | 89234  | May 15 11:22 |
| sudo systemctl status |       |                  |            |            |        |              |

## Notes

- This command is designed to run with sudo for complete information; running without sudo will show incomplete results
- The 'sudo bash -c' wrapper preserves the entire command context and allows proper variable expansion
- Disk quota requires quota system to be enabled on the filesystem; shows 'none' if unavailable
- File count calculation uses 'find' which can be slow on directories with millions of files
- Account creation date uses home directory birth time (%w) if available, falling back to modification time (%y)
- Last command is truncated to 30 characters; extracted from .bash\_history. Note: Bash only flushes history to disk upon session exit or when 'history -a' is run, so this shows the last saved command, not necessarily the active session's most recent command.
- Users with UIDs  $\geq 1000$  are system accounts and are excluded by default

- The command uses `getent` instead of `/etc/passwd` for better compatibility with different user databases (LDAP, NIS, etc.)

## Warnings

- Requires `sudo` access to function properly; user must have `sudo` privileges without password prompt configured in `sudoers` for unattended execution
- Large file counts or extensive directory trees will cause significant performance degradation
- Running with `find` on large home directories can consume considerable IO and CPU resources
- The `lslogins` command retrieves last login information depending on `wtmp`/`btmp` logs; accuracy varies by distribution logging policies
- Last command information depends on `.bash_history` being available and enabled; other shells use different history files
- Network-mounted home directories (NFS, SMB) will significantly increase command execution time
- On systems with many users, overall execution time can be substantial; consider running during off-peak hours

## See Also

- `find-largest-storage-users`
- `du-sort-largest-directories`
- `disk-space-usage-per-directory`
- `find-large-files-recursive`

## Status

reviewed

## Safety

caution

## Shell

bash

## Platforms

linux, gnu-linux

## Created At

2026-04-21

## Updated At

2026-04-21

# Chapter 4

## R

### 4.1 Csv Stats

#### Load CSV file and print dataset statistics using R

**Author:** Marcos de Carvalho

**Date:** 2026-04-25

Loads a CSV file using R's `read.csv` and prints rows, columns, total cells, file size, and object memory usage.

#### Language

rscript

#### Category

data-processing

#### Command

```
Rscript -e "args<-commandArgs(trailingOnly=TRUE);if(length(args)<1)stop('Need
 filename ');df<-read.csv(args[1]);cat('Rows:',nrow(df),'Cols:',ncol(df),'Cells
 : ',nrow(df)*ncol(df),'File size:',file.size(args[1]),'Object size:',format(
 object.size(df),units='auto'),'\\n')"
```

#### Explanation

The command uses `Rscript` to execute R code that reads a CSV file specified as argument, computes basic statistics (rows, columns, cells), gets file size via `file.size()`, and reports object memory usage via `object.size()`.

#### Tags

rscript, csv, statistics, data-analysis, memory

#### Dependencies

r-base-core

## Arguments

1. **CSV\_FILE** (Required): CSV file to analyze

## Examples

1. `Rscript -e "args<-commandArgs(trailingOnly=TRUE);if(length(args)<1)stop('Need filename');df<-read.csv(args[1],file.size(args[1]),'Object size:',format(object.size(df),units='auto'),'\n')"`  
`data.csv` - Analyze data.csv
2. `Rscript -e "args<-commandArgs(trailingOnly=TRUE);if(length(args)<1)stop('Need filename');df<-read.csv(args[1],file.size(args[1]),'Object size:',format(object.size(df),units='auto'),'\n')"`  
`large_dataset.csv 2>/dev/null` - Analyze suppressing warnings

## Output

|                                                                      |
|----------------------------------------------------------------------|
| Rows: 100 Cols: 10 Cells: 1000 File size: 12345 Object size: 78.2 Kb |
|----------------------------------------------------------------------|

## Notes

- Uses `read.csv` with default settings (`header=TRUE`, `stringsAsFactors=FALSE` in  $R_i=4.0$ )
- File size reported in bytes
- Object size includes R overhead
- For large files, consider adding options(`stringsAsFactors=FALSE`)

## Warnings

- May load entire file into memory
- Very large CSV files could exhaust memory

## See Also

- `tsv-stats`

## Status

draft

## Safety

safe

## Shell

posix

## Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

## Created At

2026-04-25

## Updated At

2026-04-25

## 4.2 Tsv Stats

### Load TSV file and print dataset statistics using R

**Author:** Marcos de Carvalho

**Date:** 2026-04-25

Loads a TSV (tab-separated) file using R's `read.delim` and prints rows, columns, total cells, file size, and object memory usage.

## Language

rscript

## Category

data-processing

## Command

```
Rscript -e "args<-commandArgs(trailingOnly=TRUE);if(length(args)<1)stop('Need
 filename');df<-read.delim(args[1]);cat('Rows:',nrow(df),'Cols:',ncol(df),'Cells
 : ',nrow(df)*ncol(df),'File size:',file.size(args[1]),'Object size:',format(
 object.size(df),units='auto'),'\\n')"
```

## Explanation

The command uses Rscript to execute R code that reads a TSV file specified as argument, computes basic statistics (rows, columns, cells), gets file size via `file.size()`, and reports object memory usage via `object.size()`. `read.delim` uses tab as separator by default.

## Tags

rscript, tsv, statistics, data-analysis, memory

## Dependencies

r-base-core

## Arguments

1. **TSV\_FILE** (Required): TSV (tab-separated) file to analyze

## Examples

1. 

```
Rscript -e "args<-commandArgs(trailingOnly=TRUE);if(length(args)<1)stop('Need filename');df<-read.csv(
 size:',file.size(args[1]),'Object size:',format(object.size(df),units='auto'),'\\{\\}n')"
```

 data.tsv - Analyze data.tsv
2. 

```
Rscript -e "args<-commandArgs(trailingOnly=TRUE);if(length(args)<1)stop('Need filename');df<-read.csv(
 size:',file.size(args[1]),'Object size:',format(object.size(df),units='auto'),'\\{\\}n')"
```

 large\_dataset.tsv 2>/dev/null - Analyze suppressing warnings

## Output

|                                                                      |
|----------------------------------------------------------------------|
| Rows: 100 Cols: 10 Cells: 1000 File size: 12345 Object size: 78.2 Kb |
|----------------------------------------------------------------------|

## Notes

- Uses read.delim with default settings (header=TRUE, sep='\\{\\}t', stringsAsFactors=FALSE in R<sub>i</sub>=4.0)
- File size reported in bytes
- Object size includes R overhead
- For large files, consider adding options(stringsAsFactors=FALSE)

## Warnings

- May load entire file into memory
- Very large TSV files could exhaust memory

## See Also

- csv-stats

## Status

draft

## Safety

safe

## Shell

posix

## Platforms

linux, gnu-linux, freebsd, openbsd, netbsd

## Created At

2026-04-25

## Updated At

2026-04-25

# Chapter 5

## Git

### 5.1 Init Repo With Gitignore

#### Initialize git repository, stage all files and create empty .gitignore

**Author:** Marcos de Carvalho

**Date:** 2026-04-25

Initializes a new git repository in current directory, stages all existing files, and creates an empty .gitignore file.

#### Language

git

#### Category

version-control

#### Command

```
git init && git add . && touch .gitignore
```

#### Explanation

This command sequence creates a new git repository (git init), adds all files in the current directory to the staging area (git add .), and creates an empty .gitignore file (touch .gitignore) for future ignore patterns.

#### Tags

git, version-control, repository, init, gitignore

#### Dependencies

git

## Arguments

1. **DIRECTORY** (Optional): Directory where to initialize repository  
Default: .

## Examples

1. `git init && git add . && touch .gitignore` - Initialize repository in current directory
2. `cd /path/to/project && git init && git add . && touch .gitignore` - Initialize repository in specific directory

## Output

```
Initialized empty Git repository in /home/user/project/.git/
```

## Notes

- The .gitignore file is empty after creation; you should add patterns to ignore unnecessary files
- If .gitignore already exists, touch command updates its timestamp
- Use git status to verify files are staged

## Warnings

- `git add .` stages all files including potentially sensitive data
- Review staged files before committing

## See Also

- `git-commit-initial`
- `git-ignore-patterns`

## Status

reviewed

## Safety

safe

## Shell

posix

## **Platforms**

linux, gnu-linux, freebsd, openbsd, netbsd

## **Created At**

2026-04-25

## **Updated At**

2026-04-25